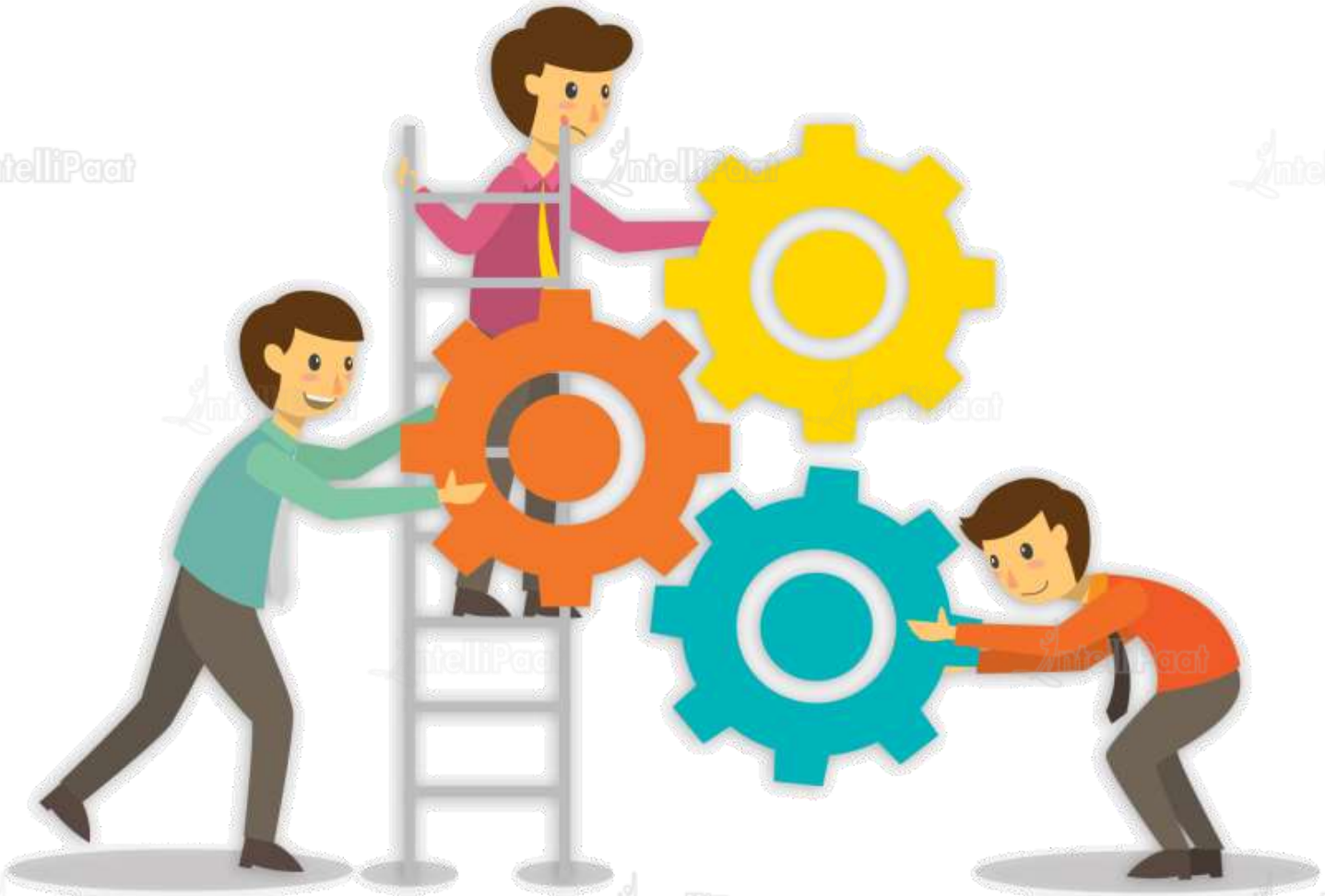
Python Training

Python Thread



Copyright Intellipaat. All rights reserved.

Agenda

01 What is a Thread?

02 Threading a Module

03 Starting a Thread

04 Daemon Threading

05 Join() in Thread

06 Using a ThreadPoolExecutor

07 One Thread

08 Two Thread

09 Producer consumer Threading

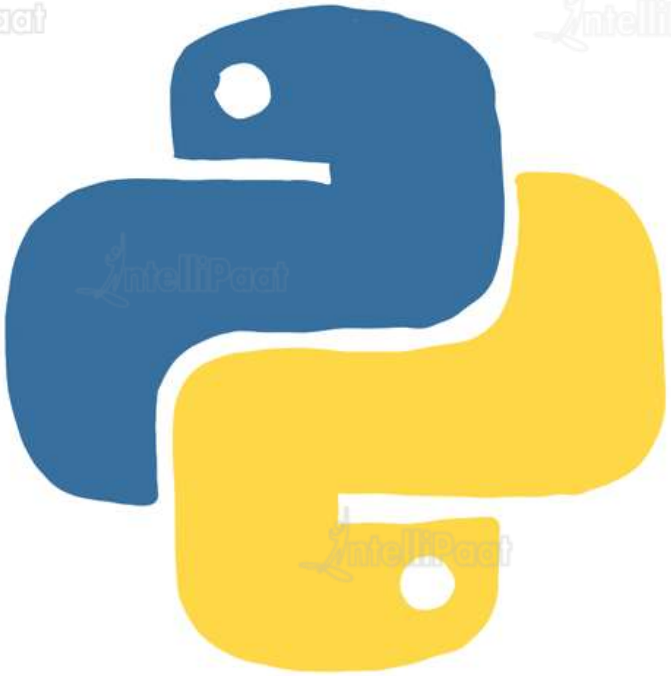
Copyright Intellipaat. All rights reserved.

What is Thread?

Copyright Intellipaat. All rights reserved.

What is Thread?

- Running several threads is similar to running several different programs concurrently, but with the following benefits –
- Multiple threads within a process share the same data space with the main thread and can therefore share information or communicate with each other more easily than if they were separate processes.
- Threads sometimes called lightweight processes and they do not require much memory overhead; they are cheaper than processes.
- A thread has a beginning, an execution sequence, and a conclusion.
- It has an instruction pointer that keeps track of where within its context it is currently running. It can be preempted (interrupted)
- It can temporarily be put on hold (also known as sleeping) while other threads are running - this is called yielding.

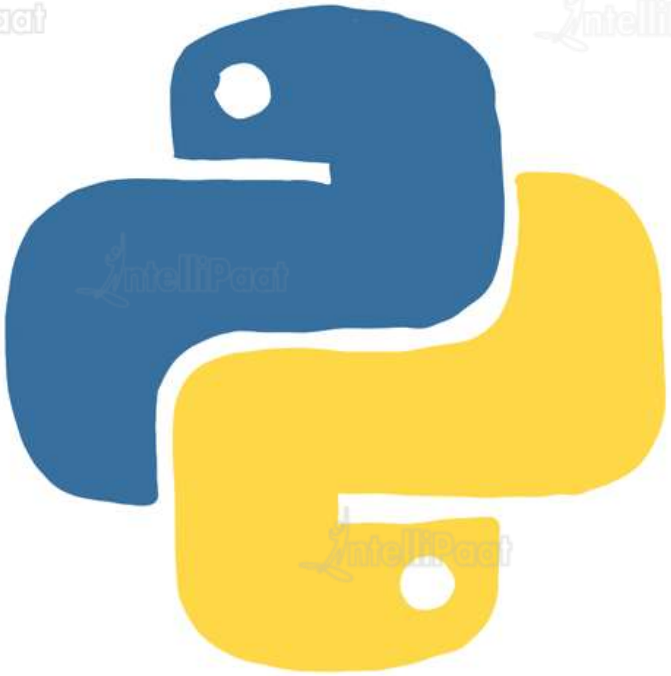


Thread

Copyright Intellipaat. All rights reserved.

What is Thread?

- A thread is a separate flow of execution.
- This means that your program will have two things happening at once.
- But for most Python 3 implementations the different threads do not actually execute at the same time: they merely appear to.
- It’s tempting to think of threading as having two (or more) different processors running on your program, each one doing an independent task at the same time.
- The threads may be running on different processors, but they will only be running one at a time.
- Getting multiple tasks running simultaneously requires a non-standard implementation of Python, writing some of your code in a different language, or using multiprocessing which comes with some extra overhead.
- Because of the way CPython implementation of Python works, threading may not speed up all tasks.

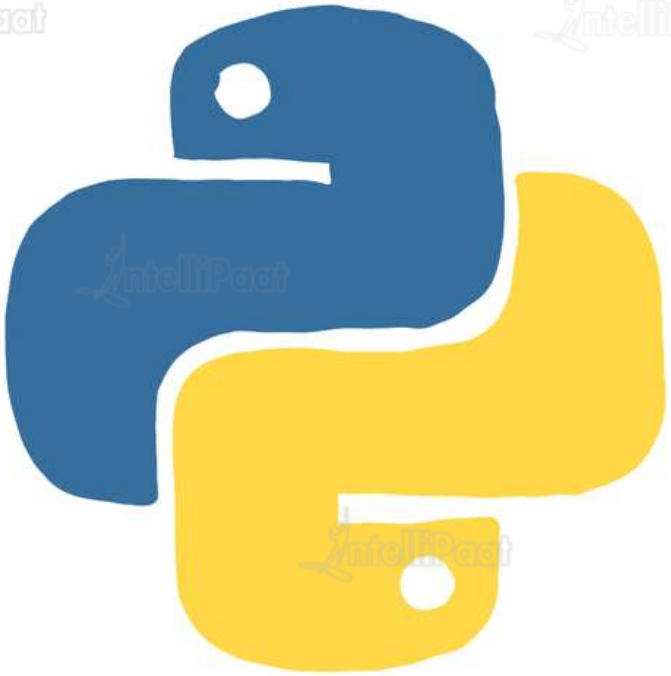


Thread

Copyright Intellipaat. All rights reserved.

What is Thread?

- This is due to interactions with the GIL that essentially limit one Python thread to run at a time.
- Tasks that spend much of their time waiting for external events are generally good candidates for threading.
- Problems that require heavy CPU computation and spend little time waiting for external events might not run faster at all.
- This is true for code written in Python and running on the standard CPython implementation.
- If your threads are written in C they have the ability to release the GIL and run concurrently.
- If you are running on a different Python implementation, check with the documentation too see how it handles threads.



Thread

Copyright Intellipaat. All rights reserved.

Thread

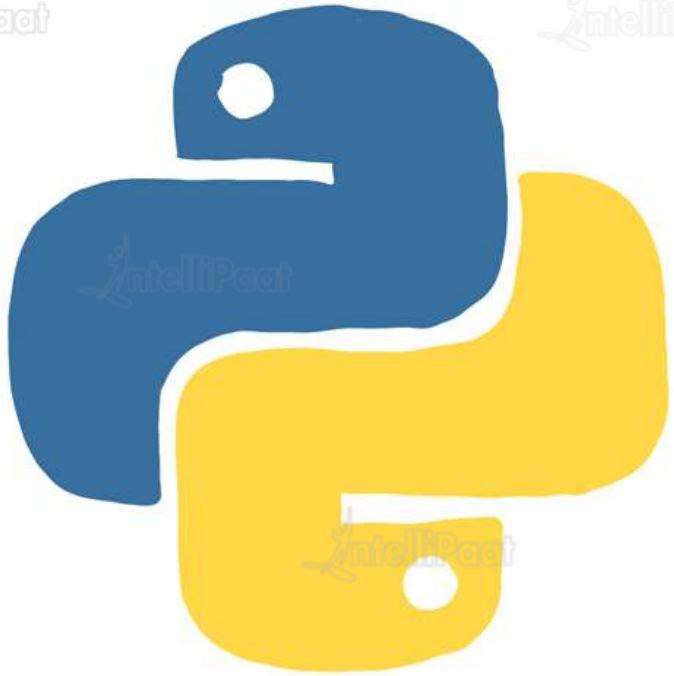
- If you look around the logging statements, you can see that the main section is creating and starting the thread:

- Python
  x = threading.Thread(target=thread_function, args=(1,))
  x.start()

- When you create a Thread, you pass it a function and a list containing the arguments to that function. In this case, you're telling the Thread to run thread_function() and to pass it 1 as an argument.

- There is threading.get_ident(), which returns a unique name for each thread, but these are usually neither short nor easily readable.

- thread_function() itself doesn't do much. It simply logs some messages with a time.sleep()in between them.

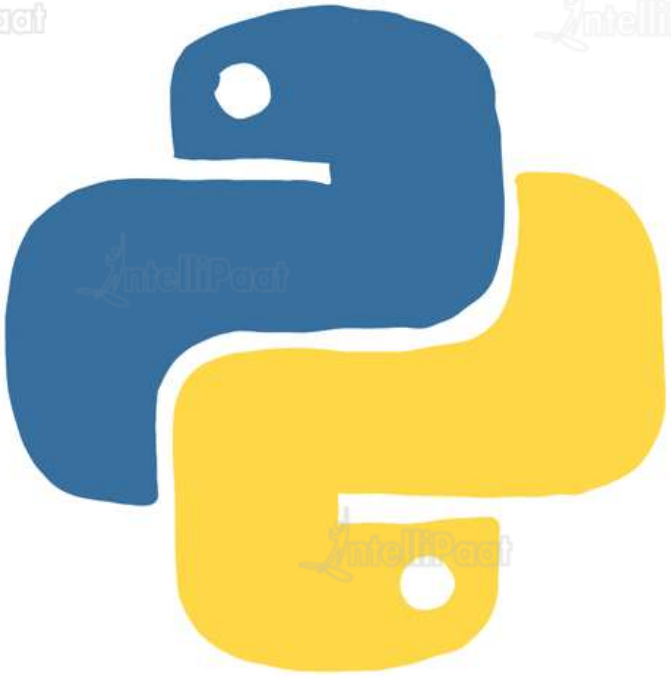


Thread

Copyright Intellipaat. All rights reserved.

Threading Modules

- The newer threading module included with Python 2.4 provides much more powerful, high-level support for threads than the thread module discussed in the previous section.
- The threading module exposes all the methods of the thread module and provides some additional methods –
- **threading.activeCount()** – Returns the number of thread objects that are active.
- **threading.currentThread()** – Returns the number of thread objects in the caller's thread control.
- **threading.enumerate()** – Returns a list of all thread objects that are currently active.
- In addition to the methods, the threading module has the Thread class that implements threading.



Thread

Copyright Intellipaat. All rights reserved.

Starting a Thread

- The Python standard library provides threading, which contains most of the primitives you'll see in this article.
- Thread, in this module, nicely encapsulates threads, providing a clean interface to work with them.
- To start a separate thread, you create a Thread instance and then tell it to .start():

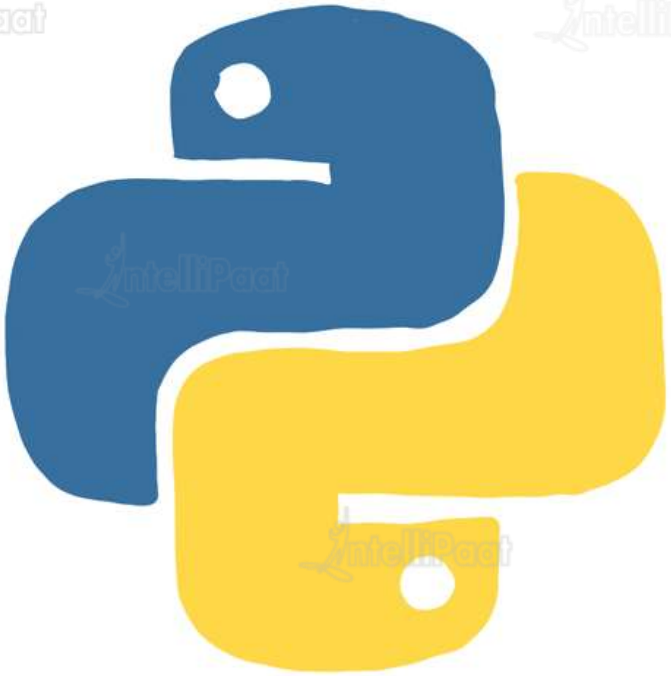
```
import logging
import threading
import time
def thread_function(name):
    logging.info("Thread %s: starting", name)
    time.sleep(2)
    logging.info("Thread %s: finishing", name)
    if __name__ == "__main__":
        format = "%(asctime)s: %(message)s"
        logging.basicConfig(format=format, level=logging.INFO,
            datefmt="%H:%M:%S")

    logging.info("Main    : before creating thread")
x = threading.Thread(target=thread_function, args=(1,))
logging.info("Main    : before running thread")
x.start()
logging.info("Main    : wait for the thread to finish")
# x.join()
logging.info("Main    : all done")
```

Copyright Intellipaat. All rights reserved.

Daemon Thread

- In computer science, a daemon is a process that runs in the background.
- Python threading has a more specific meaning for daemon.
- A daemon thread will shut down immediately when the program exits.
- One way to think about these definitions is to consider the daemon thread a thread that runs in the background without worrying about shutting it down.
- If a program is running Threads that are not daemons, then the program will wait for those threads to complete before it terminates.
- This pause is Python waiting for the non-daemon thread to complete.
- When your Python program ends, part of the shutdown process is to clean up the threading routine.

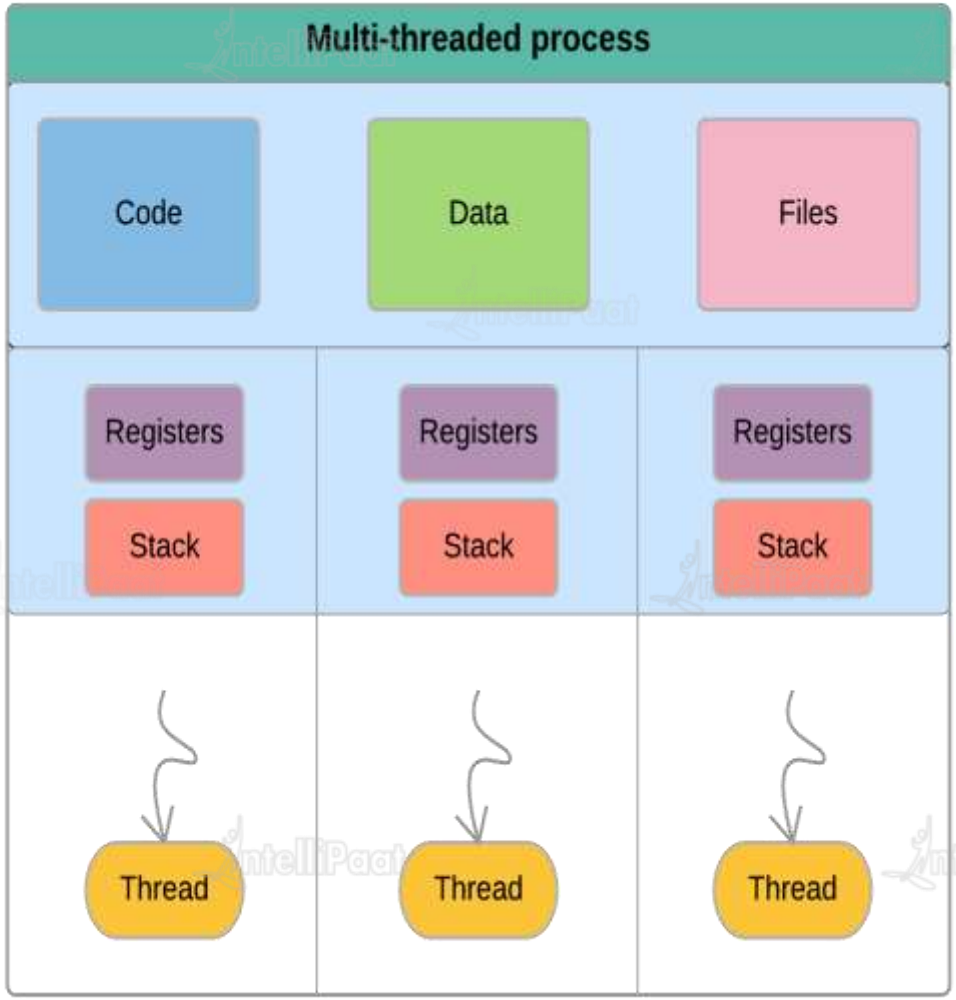


Thread

Copyright Intellipaat. All rights reserved.

Join() a Thread

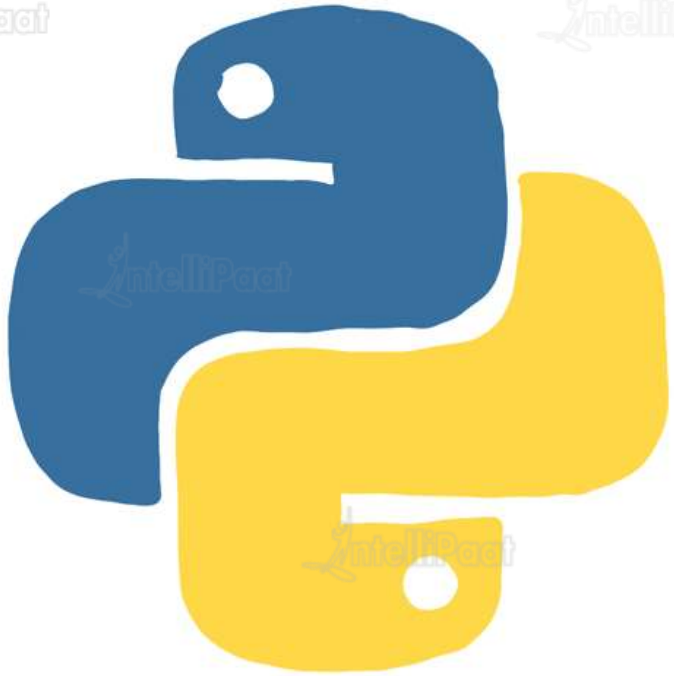
- Daemon threads are handy.
  # x.join()
- To tell one thread to wait for another thread to finish, you call
  .join().
- If you uncomment that line, the main thread will pause and wait for the thread x to complete running.
- It turns out that it doesn't matter. If you
  .join()
- A thread, that statement will wait until either kind of thread is finished.



Copyright Intellipaat. All rights reserved.

Using a ThreadPoolExecutor

- There's an easier way to start up a group of threads than the one you saw above.
- It's called a ThreadPoolExecutor, and it's part of the standard library in concurrent.futures.
- The easiest way to create it is as a context manager, using the with statement to manage the creation and destruction of the pool.
- The code creates a ThreadPoolExecutor as a context manager, telling it how many worker threads it wants in the pool.
- It then uses .map() to step through an iterable of things, in your case range(3), passing each one to a thread in the pool.
- The end of the with block causes the ThreadPoolExecutor to do a .join() on each of the threads in the pool. It is strongly recommended that you use ThreadPoolExecutor.

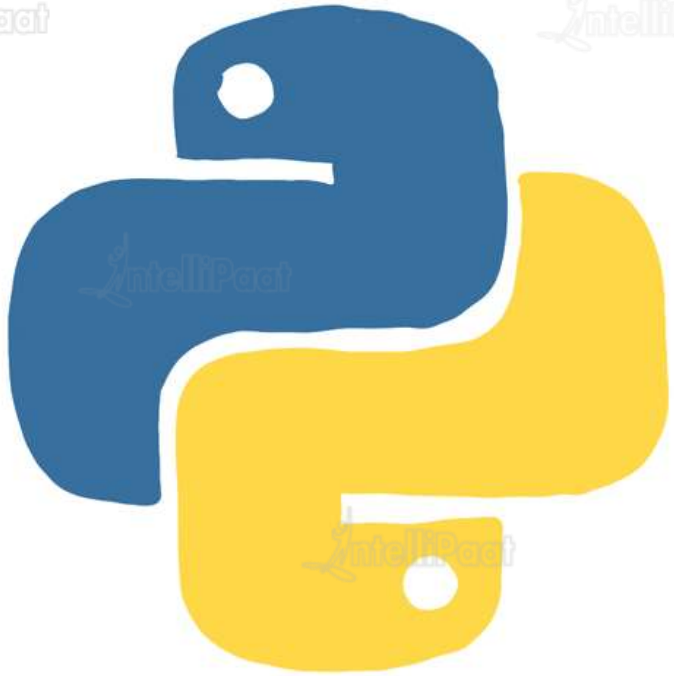


Thread

Copyright Intellipaat. All rights reserved.

One Thread

- When you tell your ThreadPoolExecutor to run each thread, you tell it which function to run and what parameters to pass to it: executor.submit(database.update, index).
- The result of this is that each of the threads in the pool will call database.update(index).
- Note that database is a reference to the one Fake Database object created in __main__. Calling .update() on that object calls an instance method on that object.
- When Thread 1 starts, Fake Database.value is zero.
- The first line of code in the method, local_copy = self.value, copies the value zero to the local variable.
- Next it increments the value of local_copy with the local_copy += 1 statement. You can see .value in Thread 1 getting set to one.

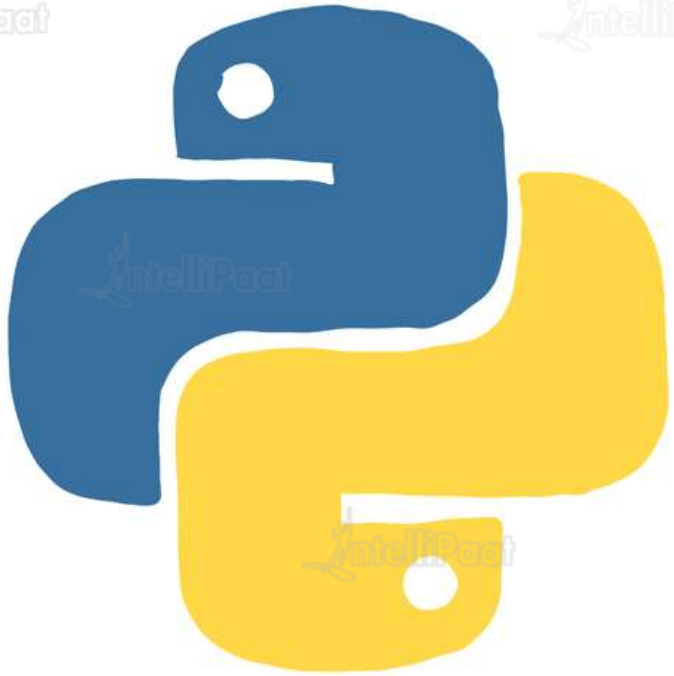


Thread

Copyright Intellipaat. All rights reserved.

Two Thread

- Getting back to the race condition, the two threads will be running concurrently but not at the same time.
- They will each have their own version of local_copy and will each point to the same database
- It is this shared database object that is going to cause the problems.
- When Thread 1 calls time.sleep(), it allows the other thread to start running.
- Thread 2 starts up and does the same operations.
- It’s also copying database.value into its private local_copy, and this shared database.
- Thread 1 now wakes up and saves its version of local_copy and then terminates, giving Thread 2 a final chance to run.

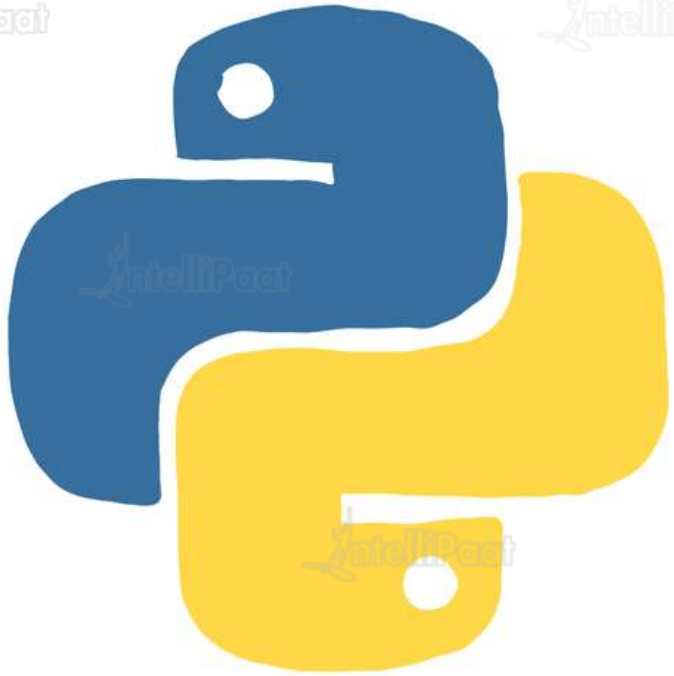


Thread

Copyright Intellipaat. All rights reserved.

Producer Consumer Thread

- Getting back to the race condition, the two threads will be running concurrently but not at the same time.
- They will each have their own version of local_copy and will each point to the same database
- It is this shared database object that is going to cause the problems.
- When Thread 1 calls time.sleep(), it allows the other thread to start running.
- Thread 2 starts up and does the same operations.
- It’s also copying database.value into its private local_copy, and this shared database.
- Thread 1 now wakes up and saves its version of local_copy and then terminates, giving Thread 2 a final chance to run.



Thread

Copyright Intellipaat. All rights reserved.

India: +91-7847955955

US: 1-800-216-8930 (TOLL FREE)

support@intellipaat.com

24/7 Chat with Our Course Advisor

Copyright Intellipaat. All rights reserved.